

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.5.1

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P1914 凯撒密码	入门	字符串、字符模拟
P2084 进制转换	入门	字符串、数学表达式构造
P2815 IPv6地址压缩	普及/提高-	字符串处理、规则模拟
P1308 统计单词数	普及	字符串、大小写处理、整词匹配

（一）凯撒密码

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1914>

知识点：字符串、字符模拟

题目简述

给出一个由小写字母组成的字符串，以及一个偏移量 n 。需要把字符串中的每个字符都向后移动 n 位，超出 z 时再从 a 继续循环。

最终输出加密后的结果字符串。

对于全部数据，字符串长度不超过 50， $1 \leq n \leq 26$ 。

1.00s，512MB。

题目分析

1. 读题

题目要求实现经典的凯撒密码。对每个字符都执行同样的平移操作，因此本质上是一个逐字符处理的模拟题。

2. 性质观察

小写字母共有 26 个，所以字符后移时天然具有模 26 循环的性质。也就是说，超过 z 后继续从 a 开始即可。

3. 程序设计思路

顺序扫描字符串中的每个字符：

1. 把当前字符转成相对 'a' 的编号；
2. 在这个编号上加上偏移量 n ；
3. 对 26 取模，保证结果仍落在小写字母范围内；
4. 再转回字符输出。

4. 数据结构与算法选择

本题只需要一个字符串数组即可，不需要额外复杂结构。算法上直接逐字符模拟最简单，也最符合题意。

5. 时空复杂度分析

- 时间复杂度： $O(len)$ ，每个字符只处理一次。
- 空间复杂度： $O(1)$ 或 $O(len)$ ，取决于是否原地修改字符串。

数据规模很小，直接实现即可稳定通过。

代码实现

洛谷 / 评测记录 / 评测详情

R281460014 记录详情

编程语言

C O2

代码长度

243B

用时

16ms

内存

824.00KB

测试点信息

源代码

测试点信息

#1

AC

3ms/816.00KB

#2

AC

3ms/824.00KB

#3

AC

4ms/664.00KB

#4

AC

3ms/816.00KB

#5

AC

3ms/788.00KB

jsj0708Baijiahui

所属题目

P1914 小书童——凯撒密码

评测状态

Accepted

提交时间

2026-06-11 17:24:47

```
#include<stdio.h>
#include<string.h>
int main()
{
    char s[60]={0};
    int n=0;
    scanf("%d",&n);
    scanf(" %s",s);
    int sz=strlen(s);
    for(int i=0;i<sz;i++)
    {
        int m=(s[i]-'a'+n)%26;
        printf("%c",m+'a');
    }
}
```

(二) 进制转换

题目信息

题目链接：<https://www.luogu.com.cn/problem/P2084>

题目简述

给出一个 M 进制数 N ，要求把它写成按位展开的形式。也就是说，如果某一位数字为 a_i ，位置对应幂次为 i ，就应写成 $a_i \times M^i$ 的形式。

所有系数为 0 的项不输出，非零项之间用 + 连接。

输入的数位很多，长度最多可达 1000，因此不应尝试把整个数直接转成十进制整数。

1.00s, 128MB。

题目分析

1. 读题

题目并不是让我们算出这个数的十进制值，而是要求输出它的按位展开表达式。所以关键是正确识别每一位数字及其对应的幂次。

2. 性质观察

对于一个长度为 len 的字符串，从左到右第 i 位对应的幂次就是 $len-1-i$ 。只要这一位不是 0，就应该把它写成一项加入答案中。

由于输入长度可达 1000，说明题目希望我们把它当作字符串处理，而不是数值处理。

3. 程序设计思路

读入进制 M 和数字串 N 后：

1. 从左到右扫描数字串；
2. 设当前字符对应的数位为 d ，幂次为 p ；
3. 如果 $d=0$ ，这一项直接跳过；
4. 如果 $d>0$ ，就输出 $d \times M^p$ 这一项；
5. 需要注意控制 + 的输出，只在非零项之间连接。

4. 数据结构与算法选择

本题最适合使用字符串。因为我们只关心每个字符代表的数位，不需要真的进行大整数运算。算法本质是一次顺序扫描加格式化输出。

5. 时空复杂度分析

- 时间复杂度： $O(len)$ ，整个数字串只扫描一遍。
- 空间复杂度： $O(1)$ 或 $O(len)$ ，主要用于存储输入字符串。

在 1000 位的规模下，这种做法非常轻松。

代码实现

洛谷 / 评测记录 / 评测详情

R281460092 记录详情

编程语言

C O2

代码长度

376B

用时

34ms

内存

1.04MB

测试点信息

源代码

测试点信息

#1	AC	#2	AC	#3	AC	#4	AC	#5	AC	#6	AC	#7	AC
3ms/1.04MB		4ms/812.00KB		3ms/1.04MB		3ms/816.00KB		3ms/812.00KB		4ms/812.00KB		3ms/816.00KB	
#8	AC	#9	AC	#10	AC								
4ms/1.04MB		3ms/788.00KB		4ms/860.00KB									

jsj0708Baijiahui

所属题目

P2084 进制转换

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:25:22

```
#include<stdio.h>
#include<string.h>
int main()
{
    int n;
    scanf("%d",&n);
    char s[1001]={0};
    scanf("%s",s);
    int sz=strlen(s);
    for(int i=0;i<sz;i++)
    {
        if((s[i]-'0')==0)
            continue;

        if(i==0)
            printf("%d*%d^%d",s[i]-'0',n,sz-i-1);
        else
            printf("+%d*%d^%d",s[i]-'0',n,sz-i-1);
    }
}
```

(三) IPv6地址压缩

题目信息

题目链接：<https://www.luogu.com.cn/problem/P2815>

知识点：字符串处理、规则模拟

题目简述

输入一个完整写法的 IPv6 地址，共有 8 组十六进制数，每组恰好 4 位，用冒号分隔。

需要按照题目给定的规则把它压缩：每组前导零可以省略；连续的全零组可以用一次 `::` 代替，且应优先压缩最长的一段，如果有多段长度相同，则取最靠前的一段。

输入保证是合法的完整 IPv6 地址。

1.00s, 128MB。

题目分析

1. 读题

本题看似是字符串输出题，实际上规则比较细，需要同时处理两类压缩：

1. 单组内部去掉前导零；
2. 多组连续 `0000` 的整体压缩。

如果细节处理不严谨，就很容易在冒号位置或边界情况上出错。

2. 性质观察

IPv6 地址固定只有 8 组，因此完全可以先把它拆成 8 段后再逐条规则处理。连续全零组的替换只允许出现一次，因此我们需要先找出“最优压缩区间”，再统一输出。

3. 程序设计思路

可以按以下步骤完成：

1. 按冒号把地址拆成 8 个字符串段；
2. 对每一段判断是否为 `0000`，并寻找最长连续全零段；
3. 若存在可压缩区间，就记录其起止位置；
4. 输出时遍历这 8 段：
5. 如果当前位置正好进入被压缩区间，就输出一次 `::`，并跳过这一整段连续零组；
6. 其他普通分组则先去掉前导零，如果整组全为零，就输出单个 `0`。

4. 数据结构与算法选择

本题只需使用字符串数组保存 8 段内容，再通过顺序扫描找最长零段即可。因为分组数量固定，直接模拟规则是最清晰可靠的做法。

5. 时空复杂度分析

- 时间复杂度： $O(1)$ ，严格来说只处理固定的 8 组，复杂度是常数级。
- 空间复杂度： $O(1)$ ，仅保存少量分组字符串和区间信息。

虽然复杂度很低，但这题真正考察的是规则细节处理能力。

代码实现

落谷 / 评测记录 / 评测详情

R281460184 记录详情

编程语言

C O2

代码长度

2.00KB

用时

32ms

内存

1.04MB

测试点信息

源代码

测试点信息

#1

AC

3ms/788.00KB

#2

AC

3ms/788.00KB

#3

AC

4ms/812.00KB

#4

AC

3ms/816.00KB

#5

AC

3ms/812.00KB

#6

AC

3ms/788.00KB

#7

AC

3ms/1.04MB

#8

AC

3ms/1.04MB

#9

AC

4ms/816.00KB

#10

AC

3ms/812.00KB

4

jsj0708Baijiahui

所属题目

P2815 IPv6地址压缩

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:26:02

```

#include <stdio.h>
#include <string.h>

int main()
{
    char input[50];
    char groups[8][5];
    int is_zero[8] = {0};

    fgets(input, sizeof(input), stdin);
    int group_idx = 0;
    char *token = strtok(input, ":");
    while (token != NULL && group_idx < 8)
    {
        int start = 0;
        while (start < 4 && token[start] == '0')
        {
            start++;
        }

        if (start == 4)
        {
            groups[group_idx][0] = '\0';
            groups[group_idx][1] = '\0';
            is_zero[group_idx] = 1;
        }
        else
        {
            int j = 0;
            while (start < 4)
            {
                groups[group_idx][j++] = token[start++];
            }
            groups[group_idx][j] = '\0';
            is_zero[group_idx] = 0;
        }
        group_idx++;
        token = strtok(NULL, ":");
    }
}

```

```

int max_len = 0;
int current_len = 0;
int start_pos = -1;

for (int i = 0; i < 8; i++)
{
    if (is_zero[i])
    {
        current_len++;
        if (current_len > max_len)
        {
            max_len = current_len;
            start_pos = i - current_len + 1;
        }
    } else
    {
        current_len = 0;
    }
}

int output_first = 1;
if (max_len >= 1)
{
    for (int i = 0; i < start_pos; i++)
    {
        if (!output_first) printf(":");
        printf("%s", groups[i]);
        output_first = 0;
    }

    printf("::");
    output_first = 1;

    for (int i = start_pos + max_len; i < 8; i++)
    {
        if (!output_first) printf(":");
        printf("%s", groups[i]);
        output_first = 0;
    }
}
else

```

```
{  
  
    for (int i = 0; i < 8; i++)  
    {  
        if (!output_first) printf(":");  
        printf("%s", groups[i]);  
        output_first = 0;  
    }  
}  
printf("\n");  
  
return 0;  
}
```

(四) 统计单词数

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1308>

知识点：字符串、大小写处理、整词匹配

题目简述

给出一个单词和一篇文章，要求统计这个单词在文章中完整出现了多少次，并输出它第一次出现的位置。

匹配时不区分大小写，但必须按“完整单词”匹配，不能把目标单词当作其他更长单词的一部分。

如果文章中没有出现该单词，则输出 -1。

1.00s, 128MB。

题目分析

1. 读题

题目有两个关键限制：一是不区分大小写，二是必须整词匹配。因此不能简单用普通子串查找直接统计，否则可能把别的单词中的一部分也误算进去。

2. 性质观察

文章中的单词之间由空格分隔，所以一个最自然的思路是先把大小写统一，再按空格识别每个完整单词。这样每次比较的都是完整单词，自然不会误匹配到更长字符串内部。

同时，第一次出现的位置要求的是原文章中的起始下标，因此在扫描时还需要记录每个单词在原串中的起点。

3. 程序设计思路

处理流程可以设计为：

1. 先把目标单词和整篇文章都统一转换为同一大小写；
2. 从左到右扫描文章，按空格切分出一个个完整单词；
3. 每识别出一个单词，就与目标单词比较；
4. 若相同，则统计次数加一；
5. 如果这是第一次匹配，还要记录该单词的起始位置；
6. 最后根据统计结果输出出现次数和第一次位置，或输出 `-1`。

4. 数据结构与算法选择

本题主要依赖字符串顺序扫描，不需要复杂的数据结构。通过统一大小写加手动分词，可以很好地满足“忽略大小写”和“整词匹配”这两个要求。

5. 时空复杂度分析

- 时间复杂度： $O(n)$ ，文章只需线性扫描一遍。
- 空间复杂度： $O(n)$ 或 $O(1)$ ，取决于是否另开字符串存储统一大小写后的结果。

在文章长度较大的情况下，线性扫描是最稳妥的做法。

洛谷 / 评测记录 / 评测详情

R281460271 记录详情

编程语言

C O2

代码长度

1.11KB

用时

53ms

内存

1.51MB

测试点信息

源代码

测试点信息

#1

AC

4ms/1.27MB

#2

AC

4ms/1.24MB

#3

AC

4ms/1.27MB

#4

AC

3ms/1.25MB

#5

AC

4ms/1.27MB

#6

AC

4ms/1.27MB

#7

AC

4ms/1.51MB

#8

AC

9ms/1.41MB

#9

AC

9ms/1.22MB

#10

AC

8ms/1.39MB

jsj0708Baijiahui

所属题目

P1308 [NOIP 2011 普及组] 统计...

评测状态

Accepted

提交时间

2026-06-11 17:26:43

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define max 1000000
int judge(char dest1[], char temp[])
{
    int sz1 = strlen(dest1);
    int sz2 = strlen(temp);
    if (sz1 != sz2)
    {
        return 0;
    }
    for (int i = 0; i < sz1; i++)
    {
        int num1 = dest1[i] - 'a';
        int num2 = dest1[i] - 'A';
        int num3 = temp[i] - 'a';
        int num4 = temp[i] - 'A';
        if (num1 != num3 && num1 != num4&&num2!=num3&&num2!=num4)
        {
            return 0;
        }
    }
    return 1;
}

int main()
{
    char dest1[10] = { 0 };
    scanf("%s", dest1);
    char str[max] = { 0 };
    getchar();
    fgets(str,sizeof(str),stdin);
    int sz = strlen(str);
    char temp[10] = {0};
    int k = 0;
    int first = 0;
    int num = 0;
    int sum = 0;
    int flat = 0;
    for (int i = 0; i < sz; i++)
    {
        if (str[i] != ' ')
        {

```

```

        temp[k++] = str[i];
    }
    else
    {
        temp[k] = '\0';
        if (judge(dest1, temp))
        {
            flat = 1;
            if (sum == 0)
            {
                first = num;
            }
            sum++;
        }
        k = 0;
        num = i + 1;
    }
}
temp[k] = '\0';
if (judge(dest1, temp))
{
    flat = 1;
    if (sum == 0)
    {
        first = num;
    }
    sum++;
}
if (flat == 0)
{
    printf("-1\n");
}
else
{
    printf("%d %d\n", sum, first);
}
}

```